Utica University | CSC 618: Information Retrieval | Fall 2026

Relevance Feedback Using the Rocchio Algorithm

Experiments on the Cranfield collection

November 2026

Source code: github.com/Atmosu/csc618-ir-evaluation-rocchio-cranfield

1 Summary

MAP, unseen docs	MRR, unseen docs	P@10, unseen docs	Queries up / down
0.1209 → 0.2643	**0.2392 → 0.4785**	**0.0950 → 0.1532**	**156 / 40**

I added Rocchio relevance feedback to my Part C search system on the Cranfield collection (Apache Lucene, TF-IDF). With feedback from the top 10 results, MAP on documents the user had not yet seen rose from 0.1209 to 0.2643. AP went up on 156 of 201 queries and down on 40. Rocchio helped most when the top 10 held at least one relevant document, and using negative feedback alone made results worse. The feedback choices came from the answer key, so these results show what happens with a user who never makes a mistake.

2 Implementation

The documents, queries, text cleaning (`EnglishAnalyzer`) and TF-IDF scoring are the same as in my Cranfield Part C: 1400 documents and 225 queries. With feedback switched off, the new code gives MAP 0.2931, the same as Part C, and the same top 10 for all 225 queries. Query numbers are matched to the judgments by position, as in Part C.

Rocchio moves the query toward relevant documents and away from non-relevant ones:

$$\tilde{q}_{new} = \alpha\,\tilde{q}_0 + \frac{\beta}{|D_r|}\sum_{\tilde{d}\in D_r}\tilde{d} \;-\; \frac{\gamma}{|D_{nr}|}\sum_{\tilde{d}\in D_{nr}}\tilde{d}$$

Here $\tilde{q}_0$ is the original query, D_r the relevant documents and D_{nr} the non-relevant documents chosen by the user.

Table 1: How each step works in the code.

Step	What the code does
Document vector	Each word gets Lucene's TF-IDF weight: $\sqrt{tf}\cdot idf\cdot 1/\sqrt{\text{length}}$. The vector is then scaled to length 1.
Query vector	Word count times idf, scaled to length 1.
Update	Applies the formula above, then removes any word whose weight is zero or negative.
Search	Sends each word to Lucene with boost = weight / idf. Lucene's score then equals query weight times document weight, summed over words.
User selection	`RocchioDemo.java` shows the top 10 and asks the user to type the ranks of relevant and non-relevant documents.
Batch runs	`RocchioExperiment.java` repeats the steps for all 225 queries, using the answer key in place of the user.
Scoring	`rocchio_eval.py` uses my Part B metric functions, unchanged.

3 Experiment method

For each query I recorded the original ranking, marked the top 10 results as relevant or non-relevant using the answer key, ran the updated query, and recorded the new ranking. The default settings were $\alpha = 1$, $\beta = 0.75$ and $\gamma = 0.15$, the values suggested by Manning et al. [1]. My copy of the Cranfield judgments is yes or no, so nDCG uses gain 1 for relevant documents.

I scored the rankings in two ways. “All documents” is the plain before-and-after comparison, but it flatters Rocchio because the user has already told the system which top documents are relevant, and those documents naturally rise. “Unseen documents” removes the 10 judged documents from both rankings and from the answer key [1], so it measures only new documents found for the user. I treat this as the fair result. It covers 201 queries, because the other 24 have no relevant document left after the top 10 is removed.

4 Results

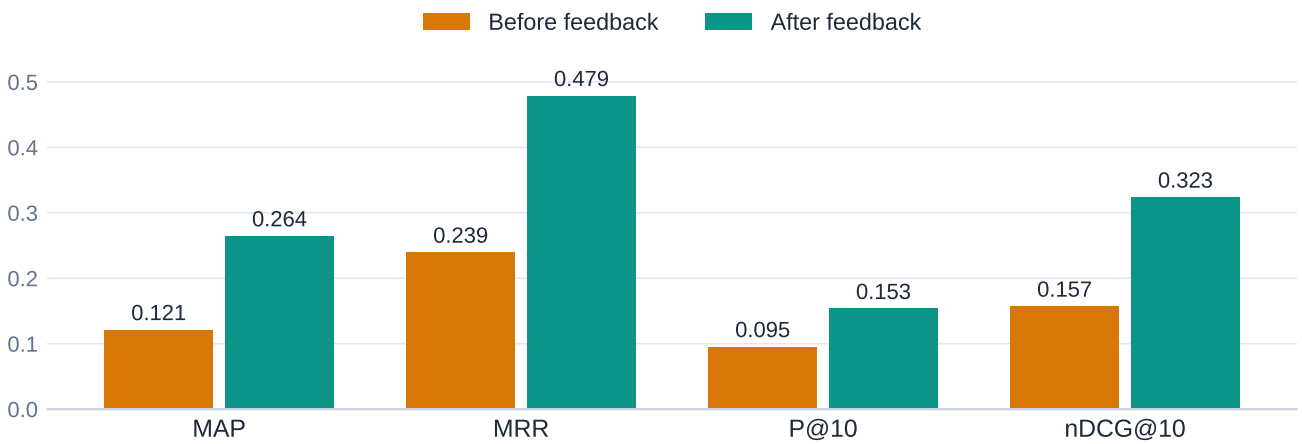


Figure 1: Average scores on unseen documents before and after feedback.

Table 2: Average scores before and after feedback (α = 1, β = 0.75, γ = 0.15).

	All documents (225 queries)			Unseen documents (201 queries)		
Metric	**Before**	**After**	**Change**	**Before**	**After**	**Change**
Precision (whole list)	0.0081	0.0071	−0.0010	0.0060	0.0055	−0.0006
Recall (whole list)	0.9513	0.9687	+0.0174	0.9374	0.9631	+0.0257
F1 (whole list)	0.0160	0.0141	−0.0019	0.0119	0.0108	−0.0011
P@3	0.3452	0.6074	+0.2622	0.0962	0.2587	+0.1625
P@10	0.2244	0.2907	+0.0662	0.0950	0.1532	+0.0582
Recall@10	0.3806	0.4740	+0.0934	0.2152	0.3577	+0.1425
F1@10	0.2544	0.3263	+0.0719	0.1177	0.1912	+0.0736
MAP	0.2931	0.4891	+0.1961	0.1209	0.2643	+0.1433
MRR	0.5327	0.8441	+0.3114	0.2392	0.4785	+0.2393
nDCG@10	0.3690	0.5637	+0.1946	0.1567	0.3234	+0.1666

Precision and F1 over the whole list barely move, because each list holds up to 1000 documents and Rocchio mostly changes their order. The metrics that look at the top of the list show the gain.

Table 3: Change on unseen documents, query by query. A small p means the change is unlikely to be chance (sign-flip test with 100,000 repeats).

Metric	Mean change	Queries up	Queries down	No change	p
AP	+0.1433	156	40	5	< 0.00001
P@10	+0.0582	81	7	113	< 0.00001
P@3	+0.1625	77	6	118	< 0.00001
nDCG@10	+0.1666	122	19	60	< 0.00001
RR	+0.2393	128	40	33	< 0.00001
Recall@10	+0.1425	81	7	113	< 0.00001

Table 4: MAP on unseen documents, grouped by how many relevant documents the original top 10 held.

Relevant in top 10	Queries	Before	After	Change	Queries down
0	34	0.0697	0.0806	+0.0109	18
1 to 2	94	0.1116	0.2721	+0.1605	17
3 to 5	58	0.1432	0.3276	+0.1844	4
6 to 10	15	0.2090	0.3860	+0.1770	1

On average, the query grew from 11 to 120 words, and no word was removed at these settings. The new top 10 shared 74% of its documents with the old one, so about 2.6 documents per query were new.

5 Demonstration

This is real output from `RocchioDemo` for query 20. I typed ranks 2 to 6 as relevant and the other five as non-relevant, following the answer key. In class the user can choose any documents. Captured runs for queries 144 and 168 are in the repository.

```
Rocchio relevance feedback on Cranfield (1400 documents, 225 judged queries)
Judged query IDs: 1 2 3 4 ... (all 225 listed)

Query ID (or q to quit): 20

1. ORIGINAL QUERY 20: has anyone formally determined the influence of joule heating, produced by the induced current, in
     magnetohydrodynamic free convection flows under general conditions .

   INITIAL RESULTS (top 10)
  1.   doc 500   joule heating in magnetohydrodynamic free-convection flows .
  2.   doc 268   several magnetohydrodynamic free-convection solutions .
  3.   doc 88    magnetohydrodynamic free-convection pipe flow .
  4.   doc 270   on combined free and forced convection laminar magnetohydro...
  5.   doc 87    free-convection magnetohydrodynamic flow past a porous flat...
  6.   doc 407   stationary convection flow of an electrically conducting li...
  7.   doc 450   some physical interpretations of magnetohydrodynamic duct f...
  8.   doc 396   variational and lagrangian thermodynamics of thermal convec...
  9.   doc 44    tip-bluntness effects on cone pressures at m=6.85 .
 10.   doc 84    experimental investigation of the downstream influence of s...

2. SELECT DOCUMENTS FOR FEEDBACK (type rank numbers, for example: 1 3 5)
   Relevant ranks: 2 3 4 5 6
   Non-relevant ranks: 1 7 8 9 10
   Relevant documents: [268, 88, 270, 87, 407]
   Non-relevant documents: [500, 450, 396, 44, 84]

   alpha [1.0]:
   beta [0.75]:
   gamma [0.15]:

3. UPDATED QUERY: alpha=1.00 beta=0.75 gamma=0.15: 16 words before, 182 after
   Word (stemmed)         Before    After  Change
   joul                   0.4690   0.4555  lowered
   convect                0.2743   0.4390  raised
   formal                 0.4008   0.4008  same
   magnetohydrodynam      0.3097   0.3859  raised
   current                0.2949   0.2828  lowered
   influenc               0.2248   0.2782  raised
   induc                  0.2447   0.2715  raised
```

```
    free                  0.1774   0.2533  raised
    produc                0.2407   0.2407  same
    heat                  0.1564   0.2007  raised
    gener                 0.1515   0.1834  raised
    determin              0.1578   0.1818  raised
    Words removed: none

4. NEW RESULTS (top 10; R = in answer key; +n = moved up n places)
   1. R doc 268     +1 several magnetohydrodynamic free-convection solutions .
   2. R doc 88      +1 magnetohydrodynamic free-convection pipe flow .
   3. - doc 500     -2 joule heating in magnetohydrodynamic free-convection flows .
   4. R doc 270     +0 on combined free and forced convection laminar magnetohydro...
   5. R doc 407     +1 stationary convection flow of an electrically conducting li...
   6. R doc 87      -1 free-convection magnetohydrodynamic flow past a porous flat...
   7. R doc 269     +5 on a laminar free-convection flow and heat transfer of elec...
   8. R doc 408     +7 on convective motion of a conducting fluid between parallel...
   9. R doc 267    +10 steady and transient free convection of an electrically con...
  10. - doc 44      -1 tip-bluntness effects on cone pressures at m=6.85 .

    ORIGINAL RESULTS WITH ANSWER KEY
   1. - doc 500   joule heating in magnetohydrodynamic free-convection flows .
   2. R doc 268   several magnetohydrodynamic free-convection solutions .
   3. R doc 88    magnetohydrodynamic free-convection pipe flow .
   4. R doc 270   on combined free and forced convection laminar magnetohydro...
   5. R doc 87    free-convection magnetohydrodynamic flow past a porous flat...
   6. R doc 407   stationary convection flow of an electrically conducting li...
   7. - doc 450   some physical interpretations of magnetohydrodynamic duct f...
   8. - doc 396   variational and lagrangian thermodynamics of thermal convec...
   9. - doc 44    tip-bluntness effects on cone pressures at m=6.85 .
  10. - doc 84    experimental investigation of the downstream influence of s...

5. BEFORE AND AFTER

All documents
  Metric               Before    After   Change
  Precision (list)     0.0090   0.0090  +0.0000
  Recall (list)        1.0000   1.0000  +0.0000
  F1 (list)            0.0178   0.0178  +0.0000
  P@3                  0.6667   0.6667  +0.0000
  P@10                 0.5000   0.8000  +0.3000
  Recall@10            0.5556   0.8889  +0.3333
  F1@10                0.5263   0.8421  +0.3158
  AP                   0.5594   0.8692  +0.3098
  RR                   0.5000   1.0000  +0.5000
  nDCG@10              0.5417   0.8825  +0.3408

Unseen documents only (selected documents removed)
  Metric               Before    After   Change
  Precision (list)     0.0040   0.0040  +0.0000
  Recall (list)        1.0000   1.0000  +0.0000
  F1 (list)            0.0080   0.0080  +0.0000
  P@3                  0.3333   1.0000  +0.6667
  P@10                 0.3000   0.4000  +0.1000
  Recall@10            0.7500   1.0000  +0.2500
  F1@10                0.4286   0.5714  +0.1429
  AP                   0.3204   1.0000  +0.6796
  RR                   0.5000   1.0000  +0.5000
  nDCG@10              0.5148   1.0000  +0.4852

Try other alpha, beta, gamma on this query? (y/n): n

Query ID (or q to quit): q
```

6 Effect of alpha, beta and gamma

I ran 115 settings. These tables show MAP on unseen documents. For comparison, MAP with no feedback is 0.1209. I chose the settings to try and read the results on the same queries, so the best values here are a guide only.

Table 5: Changing β and γ with α = 1. The number in brackets is how many of the 201 queries got a lower AP.

	γ = 0	γ = 0.15	γ = 0.25	γ = 0.5	γ = 1
β = 0	0.1209 (0)	0.1157 (123)	0.1122 (130)	0.1012 (138)	0.0838 (142)
β = 0.25	0.2117 (22)	0.2055 (43)	0.1963 (47)	0.1821 (58)	0.1494 (94)
β = 0.5	0.2438 (29)	0.2447 (41)	0.2440 (44)	0.2399 (47)	0.2057 (62)
β = 0.75	0.2649 (23)	0.2643 (40)	0.2621 (42)	0.2582 (48)	0.2489 (53)
β = 1	0.2761 (25)	0.2783 (41)	0.2772 (44)	0.2720 (47)	0.2652 (53)
β = 1.5	0.2836 (27)	0.2871 (41)	0.2875 (43)	0.2865 (45)	0.2788 (52)

Table 6: Changing α with β = 0.75 and γ = 0.15.

α	MAP (unseen)	MAP (all)	Queries up	Queries down
0	0.2759	0.5366	125	71
0.5	0.2876	0.5183	153	44
1	0.2643	0.4891	156	40
2	0.2352	0.4489	153	41

Table 7: Changing how many top results are judged. Judging more documents removes more of them from the unseen list and changes the queries that can be scored, so compare the change.

Documents judged	Queries scored	Before (unseen)	After (unseen)	Change
3	216	0.2342	0.3053	+0.0711
5	209	0.1735	0.2717	+0.0983
10	201	0.1209	0.2643	+0.1433
20	188	0.0819	0.2277	+0.1458

7 Comparison with my CISI run

Table 8: The same Rocchio code on both collections, at the default settings.

	Cranfield	CISI
Documents	1400	1460
Average words in the original query	11.4	29.4
Queries scored on unseen documents	201	76
MAP on unseen documents, before	0.1209	0.1352
MAP on unseen documents, after	0.2643	0.1901
Queries up / down	156 / 40	68 / 7

Rocchio improved both collections, with a larger gain on Cranfield. Cranfield queries are much shorter, so my guess is that the added words give them more new information. I did not test this directly. The two also differ on γ: raising it from 0 to 0.5 helped on CISI and hurt on Cranfield when α = 1 and β = 0.75, so a value that works on one collection may not work on another.

8 Analysis questions

1. How does Rocchio modify the original query?

It adds the average of the relevant document vectors and subtracts the average of the non-relevant ones. Words in relevant documents get higher weights, and words from those documents that were not in the query are added. Words common in non-relevant documents get lower weights, and they are removed if the weight falls to zero or below. For query 20, the

query grew from 16 to 182 words. “convect” rose from 0.2743 to 0.4390, “joul” fell from 0.4690 to 0.4555, and “formal” stayed the same because no feedback document contains it.

2. How does relevance feedback affect document ranking?

Documents that share words with the relevant examples move up, even if they share few words with the original query. On query 20, relevant documents the user had not seen moved up: doc 269 from rank 12 to 7, doc 408 from rank 15 to 8, doc 267 from rank 19 to 9 and doc 104 from rank 93 to 11. Across all queries, about 2.6 of the top 10 documents were new.

3. Does Rocchio improve retrieval effectiveness?

Yes on average, with correct feedback. The gain was small when the top 10 held no relevant document, since the query then gets only negative feedback, and 18 of those 34 queries got worse.

4. Support your answer with evaluation metrics.

On unseen documents, MAP rose from 0.1209 to 0.2643, MRR from 0.2392 to 0.4785, P@10 from 0.0950 to 0.1532, and nDCG@10 from 0.1567 to 0.3234. All of these changes are unlikely to be chance (p below 0.00001). On all documents, MAP rose from 0.2931 to 0.4891, but part of that comes from the judged documents moving up.

5. What happens when the values of alpha, beta, and gamma are changed?

Only the ratio between the three values matters. For example, (α, β, γ) = (0.5, 0.75, 0.5) and (1, 1.5, 1) gave the same MAP of 0.2788.

α controls how much of the original query is kept. With β and γ fixed, MAP was 0.2352 at α = 2 and 0.2876 at α = 0.5, because a large α limits what feedback can change. With α = 0 the original query is dropped, and the number of queries that got worse rose from 40 to 71.

β had the largest effect. With α = 1 and γ = 0.15, MAP rose from 0.1157 at β = 0 to 0.2871 at β = 1.5.

γ had a smaller effect, and more of it did harm. With β = 0.75, MAP fell from 0.2649 at γ = 0 to 0.2489 at γ = 1, and the number of queries that got worse rose from 23 to 53. No original query word was removed even at γ = 1. Using only negative feedback was the worst choice. All 12 settings below the no-feedback score had β = 0.

The highest MAP I saw was 0.2958 at (α, β, γ) = (0.5, 1.5, 0.5), although 52 queries got worse there.

6. Are there cases where relevance feedback reduces retrieval effectiveness? Explain why.

Yes. AP on unseen documents fell for 40 of the 201 queries at the default settings. These are the five largest drops.

Table 9: Largest drops in AP on unseen documents.

Query	Relevant in top 10	Before	After	Heaviest added words (stemmed)
168	1	1.0000	0.1429	diverg pattern converg plai singular
144	4	0.5526	0.1241	buckl compress strength unpressur data
135	4	0.6845	0.5429	time instantan deform load fail
209	2	0.2050	0.1370	compress corner sutherland sub gradient
138	1	0.0667	0.0233	intern lowest p node mode

Query 168 fell the most. Its one judged relevant document is about flow in a converging-diverging nozzle, so Rocchio added “diverg”, “pattern” and “converg”. The other relevant document, on flow through nozzles near critical speed, fell from rank 11 to rank 15. Once the top 10 is removed, this took it from first place to seventh among unseen documents, and AP fell from 1.0 to 0.1429. A single unseen document makes this score swing widely.

On query 144, the judged relevant documents are mostly about bending and buckling of cylinders, while the query asks about membrane cylinders. The added words included “buckl” and “compress”, and the unseen document “analysis of partly wrinkled membrane” fell from rank 11 to rank 28. I think this is query drift: the feedback pulled the query toward one part of its topic. Queries with no relevant document in the top 10 are also at risk, because they have only negative feedback and no good words to add.

9 Limitations

- The answer key played the user, so these results assume perfect choices from the top 10. Real users make mistakes and may mark only a few documents.
- Cranfield is small, old and about one subject, and the copy I used has yes-or-no judgments, so nDCG here does not use grades.
- Some queries have very few relevant documents left after the top 10 is removed, so one document can change a query's score a lot.
- I tested Rocchio with TF-IDF scoring only.
- The settings were compared on the same queries used to report results, with no separate test set.
- I checked the scores against the ranx library [4], which matched to six decimals. I did not measure speed.

References

[1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge Univ. Press, 2008, Ch. 9. https://nlp.stanford.edu/IR-book/html/htmledition/relevance-feedback-and-query-expansion-1.html

[2] J. J. Rocchio, "Relevance feedback in information retrieval," in *The SMART Retrieval System*, G. Salton, Ed., Prentice-Hall, 1971.

[3] Apache Lucene, https://lucene.apache.org/.

[4] E. Bassani, "ranx: A blazing-fast Python library for ranking evaluation and comparison," ECIR 2022.

[5] Cranfield collection, copy used: https://github.com/oussbenk/cranfield-trec-dataset. Original: http://ir.dcs.gla.ac.uk/resources/test_collections/cran/.

Appendix A: Code

RocchioCore.java

```
import java.io.*; import java.nio.file.*; import java.util.*;
import org.apache.lucene.analysis.*; import org.apache.lucene.analysis.en.EnglishAnalyzer;
import org.apache.lucene.analysis.tokenattributes.CharTermAttribute;
import org.apache.lucene.document.*; import org.apache.lucene.index.*; import org.apache.lucene.search.*;
import org.apache.lucene.search.similarities.ClassicSimilarity; import org.apache.lucene.store.*;

/** Shared code: builds the Part C index (Cranfield, EnglishAnalyzer, TF-IDF), holds the TF-IDF vectors,
 *  applies the Rocchio formula, and searches with a query vector. */
public class RocchioCore {
  static final String FIELD = "contents";
  final Analyzer analyzer = new EnglishAnalyzer();
  final Map<String, String> titles = new LinkedHashMap<>();
  final Map<String, String> queries = new LinkedHashMap<>();
  final Map<String, Set<String>> qrels = new HashMap<>();
  final Map<String, Map<String, Double>> docVectors = new HashMap<>();
  final Map<String, Double> idf = new HashMap<>();
  IndexReader reader; IndexSearcher searcher; StoredFields stored;

  RocchioCore(Path data) throws Exception {
    Directory dir = new ByteBuffersDirectory();
    Map<String, List<String>> tokens = new HashMap<>();
    try (IndexWriter w = new IndexWriter(dir, new IndexWriterConfig(analyzer))) {
      for (String line : Files.readAllLines(data.resolve("docs.tsv"))) {
        String[] p = line.split("\t", -1);          // id, title, abstract
        titles.put(p[0], p[1]);
        Document d = new Document();
        d.add(new StringField("id", p[0], Field.Store.YES));
        String body = p[2].isEmpty() ? p[1] : p[2];  // the abstract already opens with the title
        d.add(new TextField(FIELD, body, Field.Store.NO));
        w.addDocument(d);
        tokens.put(p[0], analyze(body));
      }
    }
    reader = DirectoryReader.open(dir);
    searcher = new IndexSearcher(reader);
    searcher.setSimilarity(new ClassicSimilarity());
    stored = reader.storedFields();
    int n = reader.numDocs();
    for (var e : tokens.entrySet())
      for (String t : e.getValue())
        idf.computeIfAbsent(t, k -> { try {
          return Math.log((n + 1.0) / (reader.docFreq(new Term(FIELD, k)) + 1.0)) + 1.0;   // Lucene's TF-IDF idf
        } catch (IOException x) { throw new RuntimeException(x); } });
    for (var e : tokens.entrySet()) {
      Map<String, Integer> tf = new HashMap<>();
      for (String t : e.getValue()) tf.merge(t, 1, Integer::sum);
      Map<String, Double> v = new HashMap<>();
      double norm = 1.0 / Math.sqrt(Math.max(1, e.getValue().size()));
      for (var f : tf.entrySet()) v.put(f.getKey(), Math.sqrt(f.getValue()) * idf.get(f.getKey()) * norm);   // Lucene's document
     weight
      docVectors.put(e.getKey(), unit(v));
    }
    for (String line : Files.readAllLines(data.resolve("queries.tsv"))) {
      String[] p = line.split("\t", -1); queries.put(p[0], p[2]);   // position ID, number in the file, text
    }
    for (String line : Files.readAllLines(data.resolve("qrels.txt"))) {
      String[] p = line.trim().split("\\s+");
      if (Integer.parseInt(p[3]) > 0) qrels.computeIfAbsent(p[0], k -> new HashSet<>()).add(p[2]);
    }
  }

  List<String> analyze(String text) throws IOException {
    List<String> out = new ArrayList<>();
    try (TokenStream ts = analyzer.tokenStream(FIELD, text)) {
      CharTermAttribute word = ts.addAttribute(CharTermAttribute.class);
      ts.reset();
      while (ts.incrementToken()) out.add(word.toString());
      ts.end();
    }
    return out;
  }

  static Map<String, Double> unit(Map<String, Double> v) {
    double s = 0; for (double x : v.values()) s += x * x;
    double len = Math.sqrt(s);
    Map<String, Double> o = new HashMap<>();
    if (len > 0) for (var e : v.entrySet()) o.put(e.getKey(), e.getValue() / len);
    return o;
```

```
  }

  /** Original query vector: count of each word times its idf, scaled to length 1. */
  Map<String, Double> queryVector(String text) throws IOException {
    Map<String, Double> v = new HashMap<>();
    for (String t : analyze(text))
      if (idf.containsKey(t)) v.merge(t, idf.get(t), Double::sum);    // words in no document are dropped
    return unit(v);
  }

  /** Rocchio: q' = alpha*q + beta*(mean of relevant docs) - gamma*(mean of non-relevant docs).
   *  Words whose final weight is zero or negative are removed. */
  Map<String, Double> rocchio(Map<String, Double> q, Collection<String> rel, Collection<String> non,
                              double alpha, double beta, double gamma) {
    Map<String, Double> o = new HashMap<>();
    q.forEach((t, w) -> o.merge(t, alpha * w, Double::sum));
    if (!rel.isEmpty()) for (String d : rel)
      docVectors.get(d).forEach((t, w) -> o.merge(t, beta * w / rel.size(), Double::sum));
    if (!non.isEmpty()) for (String d : non)
      docVectors.get(d).forEach((t, w) -> o.merge(t, -gamma * w / non.size(), Double::sum));
    o.values().removeIf(w -> w <= 1e-12);
    return o;
  }

  /** Lucene's TF-IDF score is (boost * idf) * (doc weight), so boost = query weight / idf
   *  makes the score equal to the dot product of the two vectors. */
  List<String> search(Map<String, Double> q, int top) throws IOException {
    BooleanQuery.Builder b = new BooleanQuery.Builder();
    q.entrySet().stream().sorted(Map.Entry.comparingByKey()).forEach(e -> b.add(
        new BoostQuery(new TermQuery(new Term(FIELD, e.getKey())), (float) (e.getValue() / idf.get(e.getKey()))),
        BooleanClause.Occur.SHOULD));
    List<String> out = new ArrayList<>();
    for (ScoreDoc h : searcher.search(b.build(), top).scoreDocs) out.add(stored.document(h.doc).get("id"));
    return out;
  }

  // ---- metrics for the live demo (same definitions as ir_metrics.py) ----
  static double[] metrics(List<String> r, Set<String> rel) {
    int hits = 0, h10 = 0, h3 = 0; double sumP = 0, rr = 0, dcg = 0;
    for (int i = 0; i < r.size(); i++) if (rel.contains(r.get(i))) {
      hits++; sumP += (double) hits / (i + 1);
      if (rr == 0) rr = 1.0 / (i + 1);
      if (i < 10) { h10++; dcg += 1 / log2(i + 2); }
      if (i < 3) h3++;
    }
    double idcg = 0; for (int i = 0; i < Math.min(10, rel.size()); i++) idcg += 1 / log2(i + 2);
    double n = Math.max(1, rel.size());
    double p = r.isEmpty() ? 0 : (double) hits / r.size(), R = hits / n;
    double p10 = h10 / 10.0, r10 = h10 / n;
    return new double[]{p, R, f1(p, R), h3 / 3.0, p10, r10, f1(p10, r10), sumP / n, rr, idcg > 0 ? dcg / idcg : 0};
  }
  static final String[] METRIC_NAMES = {"Precision (list)", "Recall (list)", "F1 (list)", "P@3", "P@10", "Recall@10", "F1@10", "AP",
      "RR", "nDCG@10"};
  static double f1(double p, double r) { return p + r > 0 ? 2 * p * r / (p + r) : 0; }
  static double log2(double x) { return Math.log(x) / Math.log(2); }
}
```

RocchioDemo.java

```
import java.nio.file.*; import java.util.*;

/** Interactive class demo. Pick a query, mark relevant and non-relevant results, see the updated query,
 *  the new ranking and before-and-after metrics. Usage: RocchioDemo <dataDir> */
public class RocchioDemo {
  static final int SHOW = 10, TOP = 1000;
  static Scanner in = new Scanner(System.in);

  static String ask(String prompt) {
    System.out.print(prompt); System.out.flush();
    if (!in.hasNextLine()) return null;
    String s = in.nextLine().trim();
    if (!System.getenv().getOrDefault("DEMO_ECHO", "").isEmpty()) System.out.println(s);   // shows typed input when run from a
      script
    return s;
  }
  static double num(String prompt, double dflt) {
    while (true) {
      String s = ask(prompt + " [" + dflt + "]: ");
      if (s == null || s.isEmpty()) return dflt;
      try { double v = Double.parseDouble(s); if (v >= 0) return v; } catch (NumberFormatException e) { }
      System.out.println("   Please type a number that is 0 or more, or press Enter for " + dflt + ".");
    }
  }
```

```
  static String cut(String s, int n) { return s.length() <= n ? s : s.substring(0, n - 3) + "..."; }
  static void show(RocchioCore c, List<String> r, List<String> old, Set<String> rel, boolean key) {
    for (int i = 0; i < Math.min(SHOW, r.size()); i++) {
      String d = r.get(i), move = "";
      if (old != null) { int p = old.indexOf(d); move = p < 0 ? "  new" : String.format("%+5d", p - i); }
      System.out.printf("%3d. %s doc %-5s%s %s%n", i + 1, key ? (rel.contains(d) ? "R" : "-") : " ", d, move, cut(c.titles.get(d),
     62));
    }
  }
  static List<String> pick(String prompt, List<String> r, Collection<String> taken) {
    while (true) {
      List<String> out = new ArrayList<>();
      String s = ask(prompt);
      if (s == null || s.isEmpty()) return out;
      try {
        for (String n : s.split("[ ,]+")) {
          int rank = Integer.parseInt(n);
          if (rank < 1 || rank > SHOW) throw new NumberFormatException();
          String d = r.get(rank - 1);
          if (!out.contains(d) && !taken.contains(d)) out.add(d);
        }
        return out;
      } catch (NumberFormatException e) { System.out.println("   Please type rank numbers from 1 to " + SHOW + ", separated by
     spaces."); }
    }
  }
  static void compare(String label, List<String> before, List<String> after, Set<String> rel) {
    double[] a = RocchioCore.metrics(before, rel), b = RocchioCore.metrics(after, rel);
    System.out.println("\n" + label);
    System.out.printf("  %-18s %8s %8s %8s%n", "Metric", "Before", "After", "Change");
    for (int i = 0; i < a.length; i++)
      System.out.printf("  %-18s %8.4f %8.4f %+8.4f%n", RocchioCore.METRIC_NAMES[i], a[i], b[i], b[i] - a[i]);
  }

  public static void main(String[] x) throws Exception {
    java.util.logging.Logger.getLogger("org.apache.lucene").setLevel(java.util.logging.Level.OFF);
    RocchioCore c = new RocchioCore(Paths.get(x[0]));
    List<String> judged = new ArrayList<>(c.qrels.keySet());
    judged.sort(Comparator.comparingInt(Integer::parseInt));
    System.out.println("Rocchio relevance feedback on Cranfield (" + c.titles.size() + " documents, " + judged.size() + " judged
     queries)");
    System.out.println("Judged query IDs: " + String.join(" ", judged));
    while (true) {
      String id = ask("\nQuery ID (or q to quit): ");
      if (id == null || id.equalsIgnoreCase("q")) break;
      if (!c.qrels.containsKey(id)) { System.out.println("That query has no judgments. Pick one from the list."); continue; }
      Set<String> rel = c.qrels.get(id);
      System.out.println("\n1. ORIGINAL QUERY " + id + ": " + cut(c.queries.get(id), 300));
      Map<String, Double> q0 = c.queryVector(c.queries.get(id));
      List<String> r0 = c.search(q0, TOP + SHOW);
      System.out.println("\n   INITIAL RESULTS (top " + SHOW + ")");
      show(c, r0, null, rel, false);

      System.out.println("\n2. SELECT DOCUMENTS FOR FEEDBACK (type rank numbers, for example: 1 3 5)");
      List<String> good = pick("   Relevant ranks: ", r0, List.of());
      List<String> bad = pick("   Non-relevant ranks: ", r0, good);
      System.out.println("   Relevant documents: " + good);
      System.out.println("   Non-relevant documents: " + bad);

      while (true) {
        double a = num("\n   alpha", 1.0), b = num("   beta", 0.75), g = num("   gamma", 0.15);
        Map<String, Double> q1 = c.rocchio(q0, good, bad, a, b, g);
        System.out.printf("%n3. UPDATED QUERY: alpha=%.2f beta=%.2f gamma=%.2f: %d words before, %d after%n", a, b, g, q0.size(), q1
     .size());
        System.out.printf("   %-20s %8s %8s  %s%n", "Word (stemmed)", "Before", "After", "Change");
        q1.entrySet().stream().sorted((p, q) -> Double.compare(q.getValue(), p.getValue())).limit(12).forEach(e -> {
          double w0 = q0.getOrDefault(e.getKey(), 0.0);
          System.out.printf("   %-20s %8.4f %8.4f  %s%n", e.getKey(), w0, e.getValue(), w0 == 0 ? "added" : e.getValue() > w0 ? "
     raised" : e.getValue() < w0 ? "lowered" : "same");
        });
        List<String> gone = new ArrayList<>(q0.keySet()); gone.removeAll(q1.keySet());
        System.out.println("   Words removed: " + (gone.isEmpty() ? "none" : gone));

        List<String> r1 = c.search(q1, TOP + SHOW);
        System.out.println("\n4. NEW RESULTS (top " + SHOW + "; R = in answer key; +n = moved up n places)");
        show(c, r1, r0, rel, true);
        System.out.println("\n   ORIGINAL RESULTS WITH ANSWER KEY");
        show(c, r0, null, rel, true);

        System.out.println("\n5. BEFORE AND AFTER");
        compare("All documents", r0.subList(0, Math.min(TOP, r0.size())), r1.subList(0, Math.min(TOP, r1.size())), rel);
        Set<String> seen = new HashSet<>(good); seen.addAll(bad);
        Set<String> relLeft = new HashSet<>(rel); relLeft.removeAll(seen);
```

```
        List<String> u0 = new ArrayList<>(r0), u1 = new ArrayList<>(r1);
        u0.removeAll(seen); u1.removeAll(seen);
        if (relLeft.isEmpty()) System.out.println("\nUnseen documents: no relevant documents are left to measure.");
        else compare("Unseen documents only (selected documents removed)", u0.subList(0, Math.min(TOP, u0.size())), u1.subList(0,
     Math.min(TOP, u1.size())), relLeft);

        String again = ask("\nTry other alpha, beta, gamma on this query? (y/n): ");
        if (again == null || !again.equalsIgnoreCase("y")) break;
      }
    }
  }
}
```

RocchioExperiment.java

```
import java.io.*; import java.nio.file.*; import java.util.*;

/** Batch experiment. The answer key plays the user: among the top k results, documents in the key are
 *  marked relevant and the rest non-relevant. Usage: RocchioExperiment <dataDir> <resultsDir> <gridDir> */
public class RocchioExperiment {
  static final int TOP = 1020;   // leaves 1000 results after removing up to 20 seen documents

  static void run(RocchioCore c, List<String> qids, Map<String, List<String>> base, Path file, String tag,
                  double a, double b, double g, int k, PrintWriter changes) throws Exception {
    try (PrintWriter w = new PrintWriter(file.toFile())) {
      for (String q : qids) {
        List<String> seen = base.get(q).subList(0, Math.min(k, base.get(q).size()));
        List<String> rel = new ArrayList<>(), non = new ArrayList<>();
        for (String d : seen) (c.qrels.get(q).contains(d) ? rel : non).add(d);
        Map<String, Double> q0 = c.queryVector(c.queries.get(q));
        Map<String, Double> q1 = c.rocchio(q0, rel, non, a, b, g);
        List<String> r = c.search(q1, TOP);
        for (int i = 0; i < r.size(); i++) w.printf("%s Q0 %s %d 0 %s%n", q, r.get(i), i + 1, tag);
        if (changes != null) {
          long added = q1.keySet().stream().filter(t -> !q0.containsKey(t)).count();
          long removed = q0.keySet().stream().filter(t -> !q1.containsKey(t)).count();
          StringBuilder top = new StringBuilder();
          q1.entrySet().stream().filter(e -> !q0.containsKey(e.getKey())).sorted((x, y) -> Double.compare(y.getValue(), x.getValue()
     )).limit(8)
            .forEach(e -> top.append(String.format(Locale.ROOT, "%s:%.3f ", e.getKey(), e.getValue())));
          changes.printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\t%s%n", q, rel.size(), non.size(), q0.size(), q1.size(), added, removed, top.
     toString().trim());
        }
      }
    }
  }

  public static void main(String[] x) throws Exception {
    java.util.logging.Logger.getLogger("org.apache.lucene").setLevel(java.util.logging.Level.OFF);
    RocchioCore c = new RocchioCore(Paths.get(x[0]));
    Path res = Paths.get(x[1]), grid = Paths.get(x[2]);
    List<String> qids = new ArrayList<>(c.qrels.keySet());
    qids.sort(Comparator.comparingInt(Integer::parseInt));

    Map<String, List<String>> base = new HashMap<>();
    try (PrintWriter w = new PrintWriter(res.resolve("rocchio_original.run").toFile())) {
      for (String q : qids) {
        List<String> r = c.search(c.queryVector(c.queries.get(q)), TOP);
        base.put(q, r);
        for (int i = 0; i < r.size(); i++) w.printf("%s Q0 %s %d 0 original%n", q, r.get(i), i + 1);
      }
    }
    try (PrintWriter ch = new PrintWriter(res.resolve("query_changes.tsv").toFile())) {
      ch.println("qid\trel_in_top10\tnon_in_top10\torig_terms\tnew_terms\tadded\tremoved\ttop_added");
      run(c, qids, base, res.resolve("rocchio_updated.run"), "a1_b0.75_g0.15", 1, .75, .15, 10, ch);
    }
    try (PrintWriter ch = new PrintWriter(res.resolve("query_changes_gamma1.tsv").toFile())) {
      ch.println("qid\trel_in_top10\tnon_in_top10\torig_terms\tnew_terms\tadded\tremoved\ttop_added");
      run(c, qids, base, grid.resolve("gamma1_check.tmp"), "gamma1", 1, .75, 1, 10, ch);
    }
    double[] A = {0, .5, 1, 2}, B = {0, .25, .5, .75, 1, 1.5}, G = {0, .15, .25, .5, 1};
    for (double a : A) for (double b : B) for (double g : G) {
      if (a == 0 && b == 0) continue;     // would leave an empty query
      String tag = String.format(Locale.ROOT, "a%s_b%s_g%s_k10", a, b, g);
      run(c, qids, base, grid.resolve(tag + ".run"), tag, a, b, g, 10, null);
    }
    for (int k : new int[]{3, 5, 20}) {
      String tag = String.format(Locale.ROOT, "a1.0_b0.75_g0.15_k%d", k);
      run(c, qids, base, grid.resolve(tag + ".run"), tag, 1, .75, .15, k, null);
    }
  }
}
```

rocchio_eval.py

```
"""Score the original and updated rankings with the Part B metric functions (unchanged).
Usage: rocchio_eval.py <data_dir> <results_dir> <grid_dir>"""
import json, os, re, sys, collections
import numpy as np
from ir_metrics import *

def load_rel(path):
    rel = collections.defaultdict(set)
    for l in open(path):
        q, _, d, r = l.split()
        if int(r) > 0: rel[q].add(d)
    return rel
def load_run(path):
    run = collections.defaultdict(list)
    for l in open(path):
        p = l.split(); run[p[0]].append(p[2])
    return run
def score(r, rel):
    gains = {d: 1 for d in rel}
    return dict(P=precision(r, rel), R=recall(r, rel), F1=f1(r, rel), P3=precision_at_k(r, rel, 3), P10=precision_at_k(r, rel, 10),
                R10=recall(r[:10], rel), F1_10=f1(r[:10], rel), AP=average_precision(r, rel), RR=reciprocal_rank(r, rel),
                nDCG5=ndcg_at_k(r, gains, 5), nDCG10=ndcg_at_k(r, gains, 10))
def evaluate(run, base, REL, k, residual):
    """residual=True removes the top k original results (the ones the user saw) from both the list and the answer key."""
    out = {}
    for q in REL:
        seen = set(base[q][:k]) if residual else set()
        rel = REL[q] - seen
        if not rel: continue
        out[q] = score([d for d in run[q] if d not in seen][:1000], rel)
    return out
def agg(per): return {m: mean(per[q][m] for q in per) for m in next(iter(per.values()))}
def flip(x, y, rng, n=100000):
    d = np.array(x) - np.array(y); s = rng.choice([-1, 1], size=(n, len(d)))
    return float((np.abs((s * d).mean(axis=1)) >= abs(d.mean()) - 1e-12).mean())

data, res, grid = sys.argv[1:4]
REL = load_rel(f"{data}/qrels.txt")
base = load_run(f"{res}/rocchio_original.run"); upd = load_run(f"{res}/rocchio_updated.run")
out = {}

# Does the no-feedback baseline reproduce Part C's TF-IDF run?
if len(sys.argv) > 4 and os.path.exists(sys.argv[4]):
    pc = load_run(sys.argv[4]); same = 0; total = 0
    for q in REL:
        same += base[q][:1000] == pc[q][:1000]; total += 1
    out["baseline_vs_partc"] = dict(identical_queries=same, total=total, map_baseline=mean(average_precision(base[q][:1000], REL[q])
      for q in REL), map_partc=mean(average_precision(pc[q], REL[q]) for q in REL))

F0, F1_ = evaluate(base, base, REL, 10, False), evaluate(upd, base, REL, 10, False)
R0, R1_ = evaluate(base, base, REL, 10, True), evaluate(upd, base, REL, 10, True)
out["full"] = dict(before=agg(F0), after=agg(F1_), n=len(F0))
out["residual"] = dict(before=agg(R0), after=agg(R1_), n=len(R0))

rng = np.random.default_rng(618); tests = []
for name, A, B in [("full", F0, F1_), ("residual", R0, R1_)]:
    for m in ["AP", "P10", "P3", "nDCG10", "RR", "R10"]:
        qs = sorted(A, key=int); x = [B[q][m] for q in qs]; y = [A[q][m] for q in qs]; d = np.array(x) - np.array(y)
        tests.append(dict(view=name, metric=m, diff=float(d.mean()), up=int((d > 1e-12).sum()), down=int((d < -1e-12).sum()), same=
     int((abs(d) <= 1e-12).sum()), p=flip(x, y, rng)))
out["tests"] = tests

# Query changes and per-query outcome on unseen documents
ch = {l.split("\t")[0]: l.rstrip("\n").split("\t") for l in list(open(f"{res}/query_changes.tsv"))[1:]}
rows = []
for q in sorted(R0, key=int):
    c = ch[q]
    rows.append(dict(q=q, rel_top10=int(c[1]), orig_terms=int(c[3]), new_terms=int(c[4]), added=int(c[5]), removed=int(c[6]),
      top_added=c[7], nrel=len(REL[q]), ap0=R0[q]["AP"], ap1=R1_[q]["AP"], p10_0=R0[q]["P10"], p10_1=R1_[q]["P10"], dAP=R1_[q]["AP"]
       - R0[q]["AP"]))
out["query_terms"] = dict(orig=mean(int(c[3]) for c in ch.values()), new=mean(int(c[4]) for c in ch.values()), added=mean(int(c[5])
     for c in ch.values()), removed=mean(int(c[6]) for c in ch.values()))
groups = {"0": (0, 0), "1 to 2": (1, 2), "3 to 5": (3, 5), "6 to 10": (6, 10)}
out["by_rel"] = {g: dict(n=sum(lo <= r["rel_top10"] <= hi for r in rows), dAP=mean(r["dAP"] for r in rows if lo <= r["rel_top10"] <=
      hi), ap0=mean(r["ap0"] for r in rows if lo <= r["rel_top10"] <= hi), ap1=mean(r["ap1"] for r in rows if lo <= r["rel_top10"]
     <= hi), down=sum(lo <= r["rel_top10"] <= hi and r["dAP"] < 0 for r in rows)) for g, (lo, hi) in groups.items()}
cg = [l.split("\t") for l in list(open(f"{res}/query_changes_gamma1.tsv"))[1:]]
out["removed_gamma1"] = dict(mean=mean(int(c[6]) for c in cg), queries=sum(int(c[6]) > 0 for c in cg), orig=mean(int(c[3]) for c in
     cg))
s = sorted(rows, key=lambda r: r["dAP"])
out["worst"], out["best"] = s[:5], s[-5:][::-1]
```

```
out["rows"] = rows

# Ranking change
out["rank_change"] = dict(top10_overlap=mean(len(set(base[q][:10]) & set(upd[q][:10])) / 10 for q in REL),
                          new_in_top10=mean(len(set(upd[q][:10]) - set(base[q][:10])) for q in REL))

# Parameter grid
g = {}
for f in sorted(os.listdir(grid)):
    if not f.endswith(".run"): continue
    a, b, gm, k = re.match(r"a([\d.]+)_b([\d.]+)_g([\d.]+)_k(\d+)\.run", f).groups()
    run = load_run(f"{grid}/{f}"); k = int(k)
    pr = evaluate(run, base, REL, k, True); pf = evaluate(run, base, REL, k, False)
    pb = evaluate(base, base, REL, k, True)
    g[f[:-4]] = dict(alpha=float(a), beta=float(b), gamma=float(gm), k=k, residual=agg(pr), full=agg(pf), n=len(pr), base_residual=
     agg(pb),
                     down=sum(pr[q]["AP"] < pb[q]["AP"] - 1e-12 for q in pr), up=sum(pr[q]["AP"] > pb[q]["AP"] + 1e-12 for q in pr))
out["grid"] = g
json.dump(out, open(f"{res}/rocchio_results.json", "w"), indent=1)
with open(f"{res}/rocchio_per_query.csv", "w") as f:
    f.write("query,rel_in_top10,AP_before_unseen,AP_after_unseen,change,top_added_words\n")
    for r in rows: f.write(f"{r['q']},{r['rel_top10']},{r['ap0']:.6f},{r['ap1']:.6f},{r['dAP']:+.6f},{r['top_added']}\n")

for k in ["baseline_vs_partc", "query_terms", "rank_change", "by_rel"]: print(k, out.get(k))
for v in ["full", "residual"]:
    print(v, out[v]["n"]); [print(f"  {m:7} {out[v]['before'][m]:.4f} {out[v]['after'][m]:.4f}") for m in out[v]["before"]]
for t in tests: print(t)
for r in out["worst"] + out["best"]: print(r["q"], r["rel_top10"], r["nrel"], round(r["ap0"], 4), round(r["ap1"], 4), r["top_added"
     ][:70])
K = [v for v in g.values() if v["k"] == 10]
print("top residual"); [print(v["alpha"], v["beta"], v["gamma"], round(v["residual"]["AP"], 4), round(v["full"]["AP"], 4), v["up"],
     v["down"]) for v in sorted(K, key=lambda v: -v["residual"]["AP"])[:6]]
print("bottom"); [print(v["alpha"], v["beta"], v["gamma"], round(v["residual"]["AP"], 4), round(v["full"]["AP"], 4), v["up"], v["
     down"]) for v in sorted(K, key=lambda v: v["residual"]["AP"])[:5]]
for v in g.values():
    if v["k"] != 10: print("k", v["k"], v["n"], round(v["base_residual"]["AP"], 4), round(v["residual"]["AP"], 4), round(v["full"]["
     AP"], 4))
```

`ir_metrics.py` (unchanged from Part B)

```
"""IR evaluation metrics shared by Part B and Part C. A ranked list is a list of doc IDs."""
import math

def precision(ranked, rel):
    return sum(d in rel for d in ranked) / len(ranked) if ranked else 0.0

def recall(ranked, rel):
    return sum(d in rel for d in ranked) / len(rel) if rel else 0.0

def f1(ranked, rel):
    p, r = precision(ranked, rel), recall(ranked, rel)
    return 2 * p * r / (p + r) if p + r else 0.0

def precision_at_k(ranked, rel, k):
    """The denominator stays k even if fewer than k documents were returned."""
    return sum(d in rel for d in ranked[:k]) / k

def reciprocal_rank(ranked, rel):
    for i, d in enumerate(ranked, 1):
        if d in rel:
            return 1 / i
    return 0.0

def average_precision(ranked, rel):
    """Relevant documents that were not retrieved contribute 0 (division by |rel|)."""
    hits, total = 0, 0.0
    for i, d in enumerate(ranked, 1):
        if d in rel:
            hits += 1
            total += hits / i
    return total / len(rel) if rel else 0.0

def dcg(gains):
    return sum(g / math.log2(i + 1) for i, g in enumerate(gains, 1))

def ndcg_at_k(ranked, grades, k):
    """grades: doc -> gain; unjudged documents have gain 0."""
    actual = dcg([grades.get(d, 0) for d in ranked[:k]])
    ideal = dcg(sorted(grades.values(), reverse=True)[:k])
    return actual / ideal if ideal else 0.0

def mean(xs):
    xs = list(xs)
```

```
    return sum(xs) / len(xs) if xs else 0.0
```